```
[root@Ashish-LFY-Article helloworld]# lsmod
Module                  Size  Used by
ipmi_si                43275  1
ip6table_filter         2855  0
ip6_tables             19296  1 ip6table_filter
ebtable_nat             2039  0
ebtables               18401  1 ebtable_nat
ipt_MASQUERADE          2400  3
iptable_nat             6156  1
nf_nat                 23292  2 ipt_MASQUERADE,iptable_nat
nf_conntrack_ipv4       9440  4 iptable_nat,nf_nat
nf_defrag_ipv4          1449  1 nf_conntrack_ipv4
```

Figure 2: *lsmod* output

```
[root@Ashish-LFY-Article helloworld]# insmod helloworld.ko
[root@Ashish-LFY-Article helloworld]#
[root@Ashish-LFY-Article helloworld]# lsmod | grep -i helloworld
helloworld              1360  0
[root@Ashish-LFY-Article helloworld]#
[root@Ashish-LFY-Article helloworld]# modinfo helloworld.ko
filename:       helloworld.ko
version:        0.1
description:    Sample Module
author:         ASHISH BUNKAR
license:        GPL
srcversion:     7E0D036351D3DDCFF5ABFFE
depends:
vermagic:       2.6.32-131.0.15.el6.x86_64.debug SMP mod_unload modversions
parm:           hello:int
[root@Ashish-LFY-Article helloworld]#
[root@Ashish-LFY-Article helloworld]# rmmod helloworld.ko
[root@Ashish-LFY-Article helloworld]#
[root@Ashish-LFY-Article helloworld]# lsmod | grep -i helloworld
[root@Ashish-LFY-Article helloworld]#
```

Figure 3: Use of module management commands

```
/* This is the simple hello world kernel module program */
#include <linux/kernel.h>              //Contanins some often-used function prototypes
#include <linux/module.h>              //Include all kernel modules
#include <linux/sched.h>
#include <linux/moduleparam.h>         //Include the module_param() definition

MODULE_LICENSE ("GPL");                //Macro to specify the module License
MODULE_AUTHOR ("ASHISH BUNKAR");       //Macro to specify Module Aurthor
MODULE_DESCRIPTION ("Sample Module");  //Macro to give small description to module
MODULE_VERSION ("0.1");                //Macro to give the modules verison

static int count = 2;
static int i=0;
module_param(count, int , S_IRUGO);    //Macro to take runtime arguments for module

static int hello_init(void){           //Module initialization function
        for (i=0 ; i < count ; i++){
        printk(KERN_ALERT "This is my first hello world program\n");
        printk(KERN_ALERT "The current running process in the system is : \
                    \"%s\" (pid %i) \n", current->comm, current->pid);
        }
        return 0;
}

static void hello_exit(void){          //Module exit function
        printk(KERN_ALERT "Goodbye we are done\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Figure 4: Code for a simple module

```
##KDIR: place holder for kernel build directory
KDIR= /lib/modules/$(shell uname -r)/build/
## Creating the module object file
obj-m := helloworld.o

## Change the directory to KDIR and then build the module in the current working directory
default:
        make -C $(KDIR) M=`pwd` modules
clean:
        make -C $(KDIR) M=`pwd` clean
```

Figure 5: A sample makefile

hardware from an application goes via the LKM to the kernel, and then to the actual hardware (see Figure 1).

To know the list of modules running in a Linux kernel you can use the '*lsmod*' command, which actually gives the list of running modules at that point of time, by reading '*/proc/modules*' as shown in Figure 2.

Kernel modules can be broadly categorised as character, block or network modules.

Kernel module management commands

- *insmod <module-name>:* This command is to insert the new module into the kernel
- *lsmod:* This lists the modules that are currently loaded in the kernel